OWNML MACHINE LEARNING CANVAS

Designed for: lstm-cache-eviction-policy | Designed by: Michele Patella | Date: January 21, 2026 | Iteration: 3

PREDICTION TASK

What is the type of task?
Which entity are predictions made on?
What are the possible outcomes to predict?
When are outcomes observed?

TASK TYPE

Sequence prediction (i.e., predict the next access key in a sequence).

PREDICTION ENTITIES

Data keys (i.e., unique identifiers for data items).

PREDICTION OUTCOMES

Logits representing the likelihood of each key of being accessed at each future step.

OUTCOMES OBSERVATION

At the end of the predicted future time horizon.

DECISIONS

How are predictions turned into actionable recommendations or decisions for the end-user? (Mention parameters of the process / application for this.)

FROM PREDICTIONS TO DECISIONS

Model logits and variances (over a predefined time horizon) are combined into normalized eviction scores via confidence-weighted logic (for a given confidence level). These scores are used to rank cache keys, where higher scores indicate lower eviction priority, and keys with the lowest scores—excluding protected ones—are selected for eviction.

VALUE PROPOSITION

Who is the end beneficiary, and what specific pain points are addressed?
How will the ML solution integrate with their workflow, and through which user interfaces?

The primary beneficiaries are the operations/platform teams integrating the solution into a specific system (direct) and its corresponding end-users (indirect).

The key pain points addressed are:
1) Ineffective cache eviction strategies
- Rigid, rule-based policies
- Poor suitability for real-world, non-stationary data access patterns
- No use of predictive or probabilistic information
2) Inefficient resource utilization
- Low cache hit rate
- Suboptimal use of fast cache memory
3) High operational and infrastructure costs
- Increased load on backing services
- Higher infrastructure and operational expenses
4) Degraded service performance and User Experience (UX):
- Increased end-user latency
- Lower quality of service
- Negative impact on UX
- Risk of violating Service Level Objectives (SLOs)

The solution is integrated via an internal RESTful API exploiting a microservice-based architecture (gRPC for internal microservices communication). The cache management layer calls the API with recent data accesses information, as well current cache state information and optional custom API configuration, and receives the key(s) to evict whenever a new data item must be inserted into a full cache. The eviction decision is programmatic; no direct user interface is required.

DATA COLLECTION

How is the initial set of entities and outcomes sourced (e.g., database extracts, API pulls, manual labeling)?
What strategies are in place to update data continuously while controlling cost and maintaining freshness?

DATA SOURCES

Where can we get data on entities and observed outcomes? (Mention internal and external database tables or API methods.)

IMPACT SIMULATION

What are the cost/gain values for (in)correct decisions?
Which data is used to simulate pre-deployment impact?
What are the criteria for deployment?
Are there fairness constraints?

MAKING PREDICTIONS

Are predictions made in batch or in real time?
How frequently?
How much time is available for this (including featurization and decisions)?
Which computational resources are used?

Predictions are made in real-time.

The prediction's frequency is event-driven, corresponding to how often the cache requires an eviction decision (can range from highly frequent to infrequent).

The API must be low-latency; its target average latency should be under 100 milliseconds.

The API leverages CPU/GPU cores (configurable). TALK ABOUT INFRA (CONTAINER ORCHESTRATED VIA KUBERNETES, SCALABILITY OF MICROSERVICES, AND MEMORY USAGE)

BUILDING MODELS

How many models are needed in production?
When should they be updated?
How much time is available for this (including featurization and analysis)?
Which computation resources are used?

FEATURES

What representations are used for entities at prediction time?
What aggregations or transformations are applied to raw data sources?

The keys are represented by three types of input features:
- Temporal features (sine time, cosine time): capture the cyclical pattern of the time of day.
- Local features (local frequency, local recency): model short-term sequential dynamics of key usage.
- Identity feature (request): the raw key is mapped to a low-dimensional, dense vector representation via a learnable embedding layer within the model architecture.

The raw data, composed of timestamps (hours of the day) and requests (accessed keys) undergoes a preprocessing pipeline:
1) Missing values removal: raws having any missing value are removed.
2) Trigonometric time encoding: the raw timestamps are transformed into sine and cosine components for cyclical representation.
3) Local frequency: the normalized count of a key's occurences within the preceding sequence length is calculated.
4) Local recency: the normalized and reversed distance from a key's last occurence within the lookback window is calculated.

MONITORING

Which metrics and KPIs are used to track the ML solution's impact once deployed, both for end-users and for the business?
How often should they be reviewed?